PROJECT REPORT

ON

EventSpot – Event Registration System

Submitted by:
Shubham Gupta
UID: 26MCI10128

Submitted to:
Prof. Palwinder Kaur Mangat(E9422)

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

CHANDIGARH UNIVERSITY

University Institute of Computing

Punjab, India

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"EventSpot – Event Registration System"** has been prepared as an academic project using Java, JavaServer Pages (JSP), Servlets, JDBC, and MySQL. The project demonstrates web application development, database connectivity, server-side form processing, and dynamic page generation.

The application enables participants to view the available events, select an event, and submit their name, email address, and phone number through an online registration form. It displays event details such as the name, description, date, venue, and image, stores registrations in a MySQL database using JDBC, shows a confirmation message after a successful registration, and provides a page for viewing the stored registration records. The project utilizes JSP, HTML, and CSS for the user interface, Java Servlets for request processing, JDBC for database connectivity, MySQL for data storage, and Apache Tomcat 11.0.24 as the web server.

This project report is submitted in partial fulfilment of the requirements for the degree of **Master of Computer Applications (MCA)**.

Student Signature: _____ Project Supervisor: _____

Head of Department: _____ Date: _____

TABLE OF CONTENTS

Chapter 1: Introduction and Background Study 4
1.1 Project Overview 4
1.2 Problem Definition 4
1.3 Objectives and Scope 4
1.4 Technologies Used 5
1.5 Existing System and Proposed Solution 5
Chapter 2: System Requirements and Design 6
2.1 Hardware and Software Requirements 6
2.2 System Architecture 7
2.3 Data Flow and Application Workflow 7
2.4 Database Design 7
2.5 Functional and Non-Functional Requirements 9
Chapter 3: Implementation and Application Development 9
3.1 Project Structure and Application Design 9
3.2 Event Display and Event Records 10
3.3 JSP Pages and Servlets 10
3.4 JDBC Database Connectivity 11
3.5 Image Display and Form Processing 11
3.6 Input Validation and Error Handling 11
Chapter 4: Testing, Results and Conclusion 11
4.1 Testing Methodology 11
4.2 Test Cases and Validation 12
4.3 Results and Analysis 12
4.4 Limitations and Future Scope 13
4.5 Conclusion 14
4.6 Project Folder Structure 14
References 14

CHAPTER 1: INTRODUCTION AND BACKGROUND STUDY

1.1 Project Overview

EventSpot – Event Registration System is a web-based application developed using Java technologies to simplify the process of viewing events and registering for participation. Events such as workshops, seminars, competitions, training sessions, and exhibitions are regularly organized by colleges, universities, companies, and other organizations. Managing the event information and participant registrations for such activities manually can be time-consuming.

This project addresses these difficulties by providing a simple online platform. Users can view the available events, read the event details, select an event, and submit their name, email address, and phone number through a registration form. The submitted information is stored in a MySQL database and can be retrieved later for viewing.

The application uses JavaServer Pages (JSP) to generate the web pages, Java Servlets to handle requests, and JDBC to connect the Java application to MySQL. Apache Tomcat 11.0.24 is used as the web server, and the project is developed in Eclipse IDE with MySQL Workbench used for database management.

1.2 Problem Definition

Event registrations are often collected through paper forms, spreadsheets, or direct communication with the organizers. When event information and registration records are maintained separately, the process may become disorganized, and retrieving participant details can become difficult.

Common difficulties include:

- Difficulty in maintaining organized participant records.
- Time-consuming manual registration processes.
- Lack of a centralized database for event information.
- Difficulty in retrieving registration details.
- Possibility of missing information, duplicate entries, and errors during manual data entry.

The proposed system provides a web-based solution that displays event information, collects participant registrations, stores them in a structured database, and displays a confirmation message after the registration is processed.

1.3 Objectives and Scope

1.3.1 Objectives

The main objectives of the EventSpot project are:

1. To develop a web-based event registration application using Java technologies.
2. To display event information dynamically from a MySQL database.
3. To provide a registration form for collecting participant details.
4. To store registration information in MySQL using JDBC.
5. To establish communication between Java Servlets and the database.
6. To display a confirmation message after a successful registration.
7. To retrieve and display the registered participant records.
8. To understand the working of JSP, Servlets, JDBC, and Apache Tomcat.

9. To implement basic database operations in a real-world style application.
10. To develop a simple, organized, and user-friendly interface.

1.3.2 Scope of the Project

EventSpot is intended for small-scale event registration in an educational or institutional environment. It can be used to demonstrate how participants register for workshops, seminars, competitions, and other events through a web interface.

The current version includes event listing, event details display, participant registration, database storage of registrations, a registration confirmation page, and a page for displaying registration records. It does not include user login, administrator login, an administrative dashboard, online payment processing, email or SMS notifications, QR-code entry passes, automated certificate generation, or advanced analytics. These features are considered for future development.

1.4 Technologies Used

The project uses Java and several related technologies to implement the web interface, request processing, database connectivity, and data storage.

Technology / Tool	Purpose
Java 17	Main programming language for the backend logic
JSP (JavaServer Pages)	Generating dynamic web pages and displaying database content
Java Servlets	Handling HTTP requests and coordinating database operations
JDBC	Connecting the Java application to MySQL and executing SQL statements
MySQL	Relational database for event and registration records
MySQL Workbench	Creating the database, running queries, and inspecting records
HTML and CSS	Structure and visual appearance of the web pages
Eclipse IDE	Development environment for writing and organizing the project
Apache Tomcat 11.0.24	Web server and Servlet container that runs the application

1.5 Existing System and Proposed Solution

1.5.1 Existing System

In a traditional process, event information is shared through notices, posters, or messaging applications, while participants register through paper forms, spreadsheets, or other separate systems that must be maintained and searched manually.

1.5.2 Proposed Solution

EventSpot provides an integrated web interface for viewing events and submitting registrations. Event details are stored in a MySQL database and retrieved by the Java application. When a registration is submitted, a Servlet processes the request and uses JDBC to insert the information into the database, after which a confirmation page is displayed. A separate page retrieves and displays the stored registration records.

Existing System	Proposed System
Registration data may be scattered across files	Registrations are stored in one MySQL database

Retrieving participant details takes extra time	Records are retrieved with SQL queries and shown on a page
Existing System	**Proposed System**
Manual data entry may introduce errors	Participants enter their own details through a web form
Event details may need updating in several places	Event details are stored once and displayed dynamically
No single link between events and registrations	Registrations are linked to events through event_id

CHAPTER 2: SYSTEM REQUIREMENTS AND DESIGN

2.1 Hardware and Software Requirements

The application requires a computer capable of running Java, Apache Tomcat, and MySQL. The hardware figures below are reasonable development requirements rather than strict minimum specifications.

2.1.1 Hardware Requirements

Component	Recommended Requirement
Processor	Intel Core i3 or equivalent and above
RAM	8 GB recommended
Storage	5 GB or more of available space
Display	Standard laptop or desktop display
Network	Localhost operation; internet may be required to download dependencies

2.1.2 Software Requirements

Component	Requirement
Operating System	Windows
Programming Language	Java 17
Frontend	HTML, CSS, JSP
Backend	Java Servlets
Database Connectivity	JDBC
Database	MySQL
Database Management Tool	MySQL Workbench
IDE	Eclipse IDE
Web Server	Apache Tomcat 11.0.24
Web Browser	Google Chrome or another modern browser

Compatibility note: Tomcat 11 uses the Jakarta Servlet APIs. The application therefore uses compatible Jakarta imports such as jakarta.servlet.* and a compatible MySQL JDBC driver.

2.2 System Architecture

EventSpot follows a simple multi-layer web application structure in which each component performs a specific task. The browser displays the interface, the Servlets process requests, JDBC communicates with the database, and MySQL stores the data.

2.2.1 Main Components

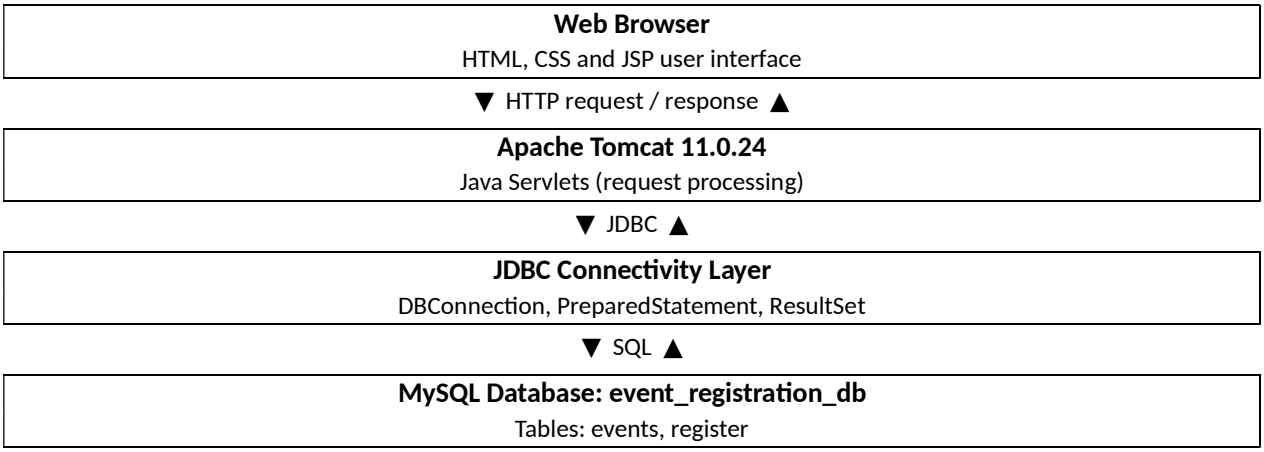
1. **Presentation Layer:** JSP and HTML pages that users interact with through a web browser. They display event information, the registration form, success messages, and registration records.

2. **Request Processing Layer:** Java Servlets that handle incoming HTTP requests, process form submissions, and coordinate the application workflow.

3. **Database Connectivity Layer:** JDBC provides communication between the Java application and MySQL. It executes SQL statements and returns query results.

4. **Database Layer:** The MySQL database stores event details and participant registration records in the events and register tables.

2.2.2 Architecture Diagram



2.3 Data Flow and Application Workflow

The application follows a sequence of operations, beginning with the display of events and ending with the confirmation of the registration.

- The participant opens the EventSpot website.
- The application retrieves the event records from MySQL and displays them as event cards.
- The participant selects an event, and the registration form opens for that event.
- The participant enters their name, email address, and phone number and submits the form.
- The Servlet receives the form data and validates the submitted information.
- JDBC executes a prepared SQL INSERT statement.
- MySQL stores the registration in the register table.

2.4 Database Design

The database used for the project is event_registration_db. It contains two main tables. One event can have multiple registrations, and each registration is linked to an event through the event_id column (a one-to-many relationship).

2.4.1 Events Table

Column	Data Type	Description
event_id	INT (Primary Key)	Unique event identifier
event_name	VARCHAR(150)	Name of the event
description	TEXT	Event description
event_date	DATE	Date of the event
venue	VARCHAR(150)	Event location
image_url	VARCHAR(255)	Relative path to the event image

2.4.2 Register Table

Column	Data Type	Description
registration_id	INT (Primary Key)	Unique registration identifier
event_id	INT (Foreign Key)	References events.event_id
name	VARCHAR(100)	Participant name
email	VARCHAR(150)	Participant email address
phone	VARCHAR(20)	Participant phone number (stored as text)
registration_date	TIMESTAMP	Registration creation time

The phone number is stored as text because phone numbers may contain a country code or leading zeros. **2.4.3 SQL Database Creation**

```
CREATE DATABASE IF NOT EXISTS event_registration_db;
USE event_registration_db;
CREATE TABLE events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
event_name VARCHAR(150) NOT NULL,
    description TEXT,
event_date DATE,     venue
VARCHAR(150),     image_url
VARCHAR(255)
);

CREATE TABLE register (
    registration_id INT AUTO_INCREMENT PRIMARY KEY,
    event_id INT NOT NULL,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL,
phone VARCHAR(20) NOT NULL,
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_register_event
        FOREIGN KEY (event_id) REFERENCES events(event_id)
);
```

The main database operations are SELECT (retrieving events and registration records), INSERT (storing a new registration), PRIMARY KEY (uniquely identifying each event and registration), and FOREIGN KEY (connecting a registration with its event).

2.5 Functional and Non-Functional Requirements

2.5.1 Functional Requirements

Functional requirements describe the operations that the application must perform.

- The application must display the available events with their names, descriptions, dates, venues, and images.
- The application must allow a participant to select an event for registration.
- The application must collect the participant's name, email address, and phone number.
- The application must store the registration in the MySQL database using JDBC.
- The application must display a confirmation message only after the registration is stored successfully.
- The application must retrieve and display the stored registration records.

2.5.2 Non-Functional Requirements

Non-functional requirements describe the expected quality and usability of the system.

- **Usability:** The interface should be easy to understand and operate.
- **Performance:** Event pages and registrations should load and process within a reasonable time.
- **Reliability:** The application should handle normal user operations and display an error message instead of falsely reporting success.
- **Maintainability:** Presentation, request processing, and database operations are kept in separate components so that each can be updated independently.
- **Portability:** The application should run on compatible systems where Java, Tomcat, and MySQL are installed.
- **Security:** Prepared statements are used to reduce SQL injection risk. The current version has no authentication and is not intended for real personal data.

CHAPTER 3: IMPLEMENTATION AND APPLICATION DEVELOPMENT

3.1 Project Structure and Application Design

The application is developed in Eclipse IDE using Java 17, with Apache Tomcat 11.0.24 configured as the server. The web application uses JSP for presentation, Servlets for request handling, and JDBC for database operations. MySQL Workbench is used to create the database, execute queries, and inspect the stored records.

3.1.1 Main Program Components

Component	Responsibility
JSP Pages	Display events, the registration form, the success message, and registration records
Servlets	Receive requests, validate input, and coordinate database operations
DBConnection	Create the JDBC connection between the application and MySQL
CSS	Style the event cards, forms, buttons, and result pages
Images	Store the event images used on the event listing page
MySQL Database	Store event details and participant registrations

3.2 Event Display and Event Records

The Event Display module retrieves event details from the events table and shows the event name, description, date, venue, and image on the main page. The listing is generated from database records rather than hardcoded HTML, so a new event added to the table appears on the page without changing the page code. Each event card provides a registration option.

The project includes the following example events and image files. The image files are stored under src/main/webapp/images/, and the database stores the matching relative path, such as images/AI.jpg.

No.	Event Name	Image Filename
1	Python Programming Workshop	ph.jpg
2	Artificial Intelligence Seminar	AI.jpg
3	Cybersecurity Awareness Workshop	cyber.jpg
4	Data Science Bootcamp	data_sc.jpg
5	Cloud Computing Workshop	c_c.jpg
6	Robotics and Automation Expo	robo.jpg
7	Entrepreneurship and Startup Meet	enter.jpg
8	Web Design and UI Workshop	wd.jpg
9	Digital Marketing Seminar	Dm.jpg
10	College Photography Competition	ph.jpg

Note: The Python Programming Workshop and the College Photography Competition currently share ph.jpg. Separate filenames can be assigned if different images are required. Filenames and capitalization must match the actual files.

3.3 JSP Pages and Servlets

3.3.1 JSP Pages

Page	Purpose and Responsibilities
index.jsp	Displays the available events with images, dates, venues, and registration links or buttons
register.jsp	Shows the selected event and provides fields for name, email, and phone; submits the data to the Servlet
success.jsp	Informs the participant that the registration succeeded; displayed only after the database insert succeeds
registrations.jsp	Displays the stored registration records in a structured format

3.3.2 Servlets

EventServlet: Establishes a database connection, executes a query to retrieve events, processes the result set, makes the event data available to the JSP, and forwards the request to the event listing page.

RegisterServlet: Receives the submitted form, reads the participant's name, email, phone, and event ID, validates the input, executes a prepared INSERT statement, checks whether the registration was stored, and then forwards or redirects the user to the appropriate result page.

ViewRegistrationsServlet: Connects to MySQL, retrieves the stored registrations, and passes the data to the registration list JSP for display.

3.4 JDBC Database Connectivity

JDBC (Java Database Connectivity) is the standard Java API for connecting to relational databases and executing SQL statements. A typical connection requires the MySQL JDBC driver, the database URL, the database username and password, and a JDBC Connection object. The connection URL follows this pattern:

```
jdbc:mysql://localhost:3306/event_registration_db
```

The username and password must match the credentials configured in the local MySQL installation. For a better implementation, database credentials should be kept outside publicly accessible web resources, and connections, statements, and result sets should be closed appropriately.

3.5 Image Display and Form Processing

3.5.1 Image Display

The event images are stored inside the web application's images directory, and the database stores relative paths such as images/AI.jpg. The JSP builds the image URL using the application's context path, which allows the images to load correctly even when the application is deployed under a context such as /EventSpot:

```
<img src="${pageContext.request.contextPath}/${event.imageUrl}"
alt="Event image">
```

3.5.2 Form Processing

When a participant submits the registration form, the browser sends the values to the Servlet named in the form's action attribute. The Servlet reads, validates, and stores them in MySQL through JDBC. The form field names must match the parameter names read by the Servlet. For example, if the Servlet uses request.getParameter("name"), the form must contain a field with name="name".

3.6 Input Validation and Error Handling

The registration Servlet should validate the submitted fields, verify that the selected event exists, and then insert the registration. Prepared statements are used for SQL operations involving user input to reduce SQL injection risk. If the registration succeeds, the success page is displayed; if an error occurs, an appropriate error message is shown instead of incorrectly reporting success.

The View Registrations module displays the participant's name, email, phone number, selected event, and registration date. Because the current version has no administrator authentication, this page is not a secure administrative interface and should not be deployed with real personal information.

CHAPTER 4: TESTING, RESULTS AND CONCLUSION

4.1 Testing Methodology

Testing is performed to verify that the application behaves as expected. It checks whether event information is displayed correctly, registrations are stored in the database, confirmation messages are shown after successful operations, and stored registration records can be retrieved. The following types of testing are used:

The main types of testing are functional testing (each feature performs its intended function), database testing (events and registrations are correctly retrieved and stored), input validation testing (missing or invalid data is

handled), integration testing (JSP pages, Servlets, JDBC, and MySQL work together), user interface testing (layout, images, forms, and messages), and error handling testing (database failures and invalid requests).

4.2 Test Cases and Validation

The following test cases can be used to evaluate the application.

Test ID	Test Scenario	Expected Result
TC01	Open the main page	Event listing page opens
TC02	Retrieve event records	Events are displayed from the database
TC03	Display an event image	Correct image is loaded
TC04	Select an event	Registration form opens for the selected event
TC05	Submit valid registration details	Registration is saved successfully
TC06	Check registration confirmation	Success message is displayed
TC07	View registration records	Stored registrations are displayed
TC08	Submit a missing required field	Invalid input is rejected or an error is shown
TC09	Submit an invalid event ID	Registration is rejected
TC10	Simulate a database connection failure	An appropriate error is handled
TC11	Inspect stored registration data	Correct values are present in the database
TC12	Open the application under its deployed context path	Pages and images load correctly

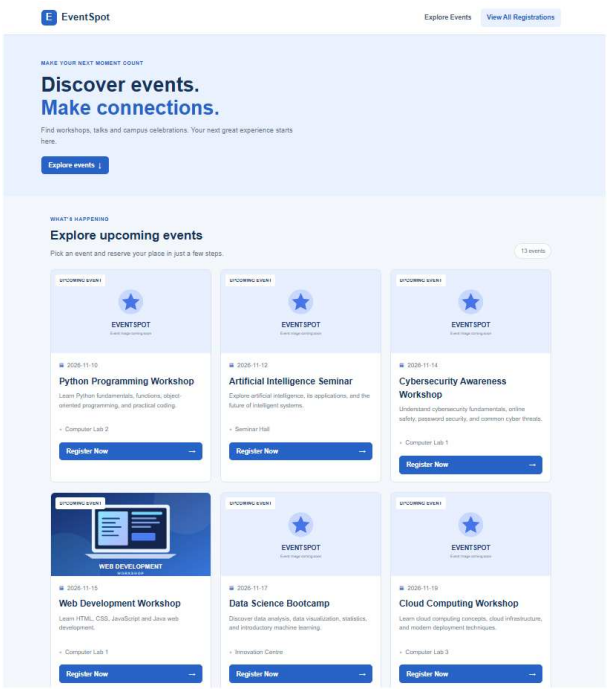
Note: These are proposed test cases. The actual result of each test should be recorded after executing it on the completed application.

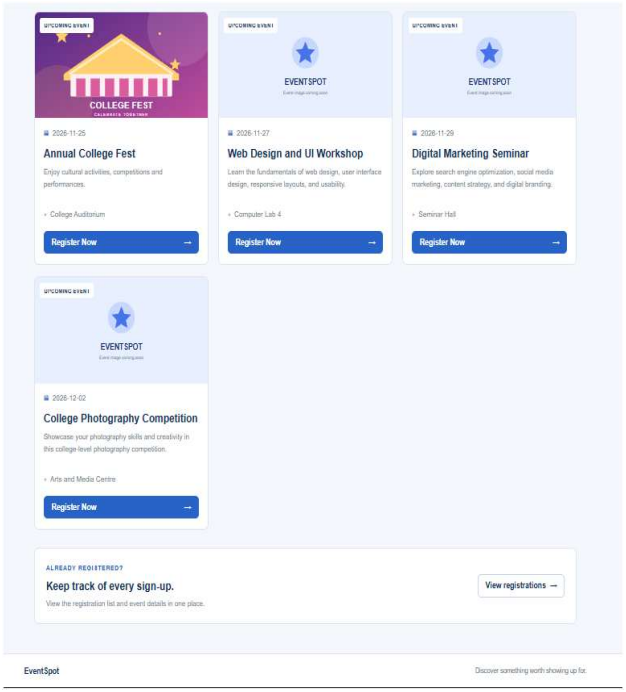
4.3 Results and Analysis

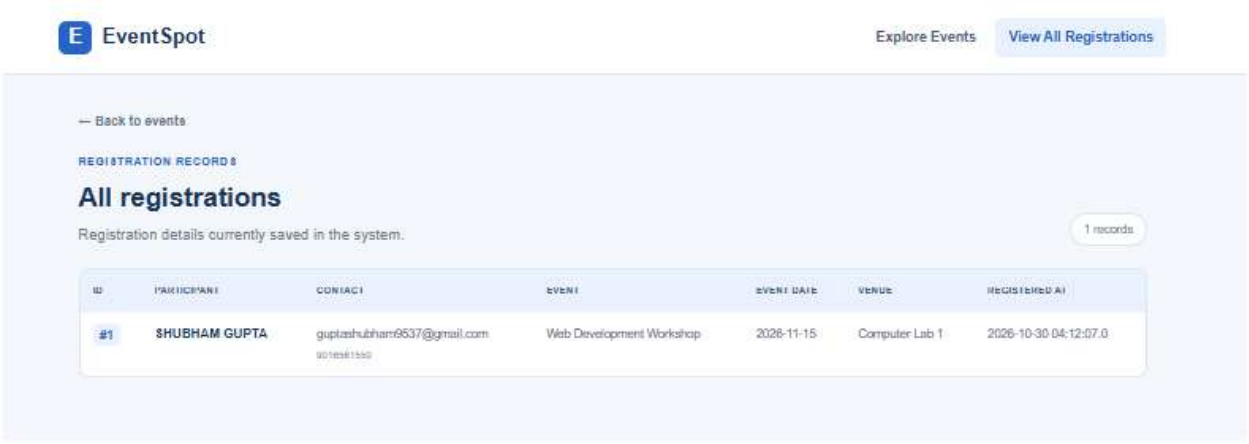
When the application runs correctly, the user can open EventSpot and view the available events retrieved from the database. Each event provides a way to proceed to registration. After selecting an event, the participant enters their details and submits the form. If the data passes validation and the database insert succeeds, a confirmation message is displayed, and the registration is stored in the register table. The registration list page can then retrieve the stored information for display.

Screenshots of the actual running application (home page, event listing, registration form, success page, registration list, and the events and register tables in MySQL Workbench) can be added to this section with figure numbers and captions.

4.3.1 System Design







4.4 Limitations and Future Scope

4.4.1 Current Limitations

- **No Authentication:** The system has no user login, individual accounts, or administrator login.
- **No Access Control:** The registration list page is not restricted to administrators.
- **No Payments or Notifications:** Online payments and email or SMS confirmations are not included.
- **No Certificates, QR Passes, or Reports:** QR-code entry passes, automatic certificates, and analytics are not provided.
- **Not Production Tested:** Suitability for large-scale use has not been established through load or security testing.

4.4.2 Future Enhancements

- **Accounts and Administration:** user login and profiles, plus an administrator dashboard for event creation, updates, and registration records.
- **Notifications and Payments:** confirmation emails, reminders, and a payment gateway for paid events.
- **QR-Code Passes and Certificates:** participant verification and automatic participation certificates.
- **Search, Duplicate Prevention, and Reports:** event search and filtering, prevention of repeated registrations, registration analytics, and stronger protection of personal information.

4.5 Conclusion

EventSpot – Event Registration System is a Java-based web application that simplifies the process of viewing events and registering for participation. It combines JSP, Servlets, JDBC, MySQL, HTML, and CSS to retrieve event information from a database, collect participant details through a registration form, store registrations, display a confirmation message, and provide a page for viewing registration records.

The project provides practical experience in Java web application development, server-side programming, relational database design, SQL operations, and database connectivity. Overall, EventSpot serves as an academic mini project that demonstrates the integration of frontend presentation, backend request processing, and database management, and it provides a foundation for future improvements such as authentication, administrator controls, notifications, payments, and analytics.

4.6 Project Folder Structure

A possible Maven-style project structure is shown below. If the actual Eclipse project uses a different layout, the real structure should be retained.

```
EventSpot/
 |-- src/main/java/com/eventspot/ (EventServlet, RegisterServlet,
 |                                 ViewRegistrationsServlet, DBConnection)
 |-- src/main/webapp/ (index.jsp, register.jsp, success.jsp, registrations.jsp,
 |                     css/style.css, images/)
 `-- pom.xml
```

GitHub Link: https://github.com/Shubhamm-9016/Event_Registration_System.git

REFERENCES

- Oracle. Java Documentation. https://docs.oracle.com/en/java/
- Jakarta EE. Jakarta Servlet Documentation. https://jakarta.ee/specifications/servlet/
- Apache Software Foundation. Apache Tomcat Documentation. https://tomcat.apache.org/
- Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/
- MySQL. MySQL Reference Manual. https://dev.mysql.com/doc/
- Eclipse Foundation. Eclipse IDE Documentation. https://help.eclipse.org/
- Mozilla. HTML Documentation. https://developer.mozilla.org/en-US/docs/Web/HTML
- Mozilla. CSS Documentation. https://developer.mozilla.org/en-US/docs/Web/CSS